Advanced Software Development

Johan Bontes (johan@digitsolutions.nl)

CSC3003S

Examples

- Builder: https://godbolt.org/z/rvMxz7q4Y
- Adapter: https://godbolt.org/z/G483PWoxa
- Decorator: https://godbolt.org/z/Ehv6Wdv1P
- Iterator: https://godbolt.org/z/e7M44ncb7
- Visitor: https://godbolt.org/z/a3raPo67P
- State: https://godbolt.org/z/3av9K56a3
- Chain of responsibility: https://godbolt.org/z/5456eG3x7
- ~~Strategy: https://godbolt.org/z/jT8Wf8cee~~
- ~~Observer: https://godbolt.org/z/q8croqeeP~~

Visitor patterns revisited

In OOP it’s easy to add types

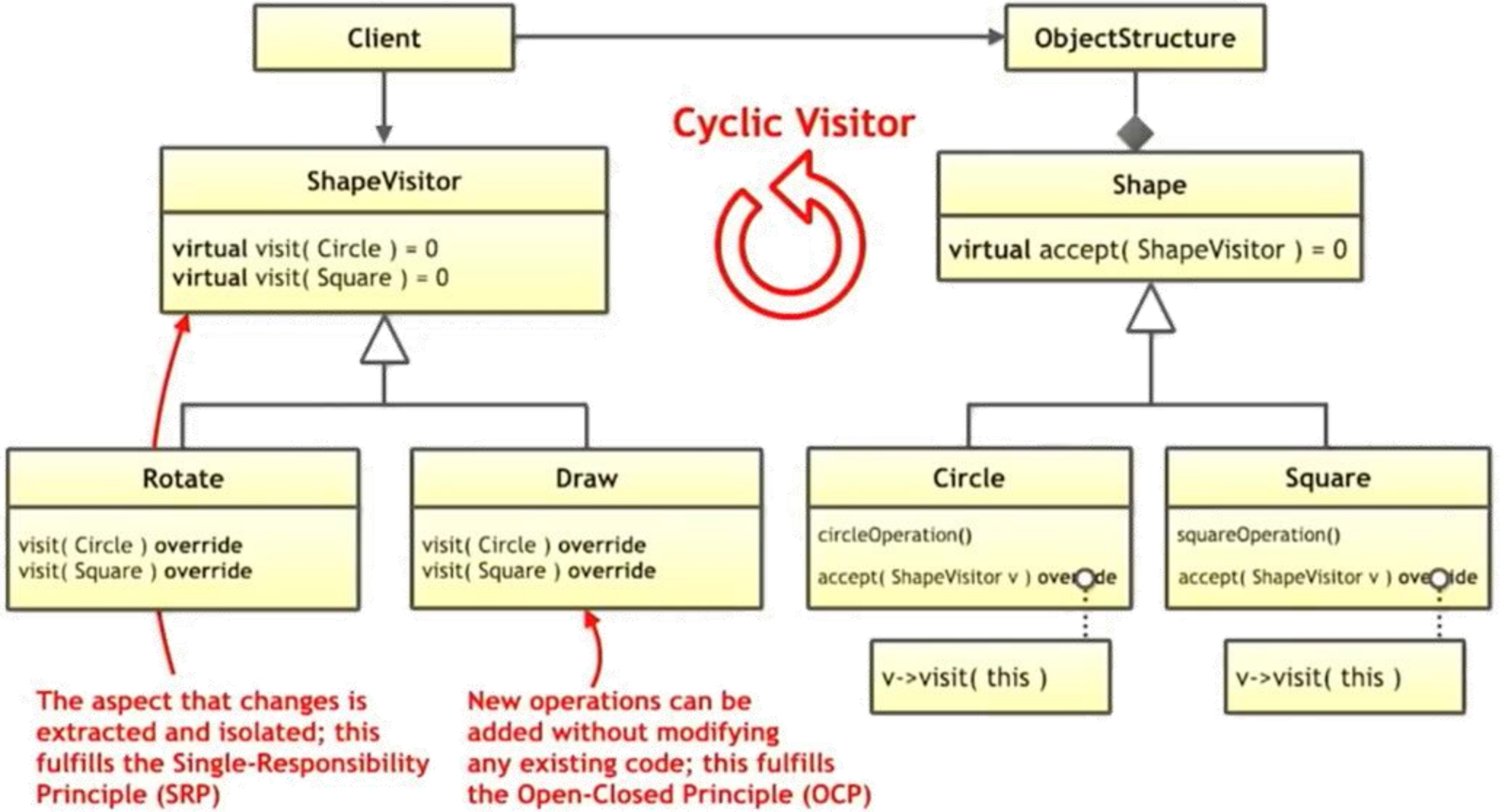
- Just add a class

But it’s hard to add operations

- Operations are another dimension
- But a hierarchy is one-dimensional

GoF visitor

Adds operations to a type hierarchy

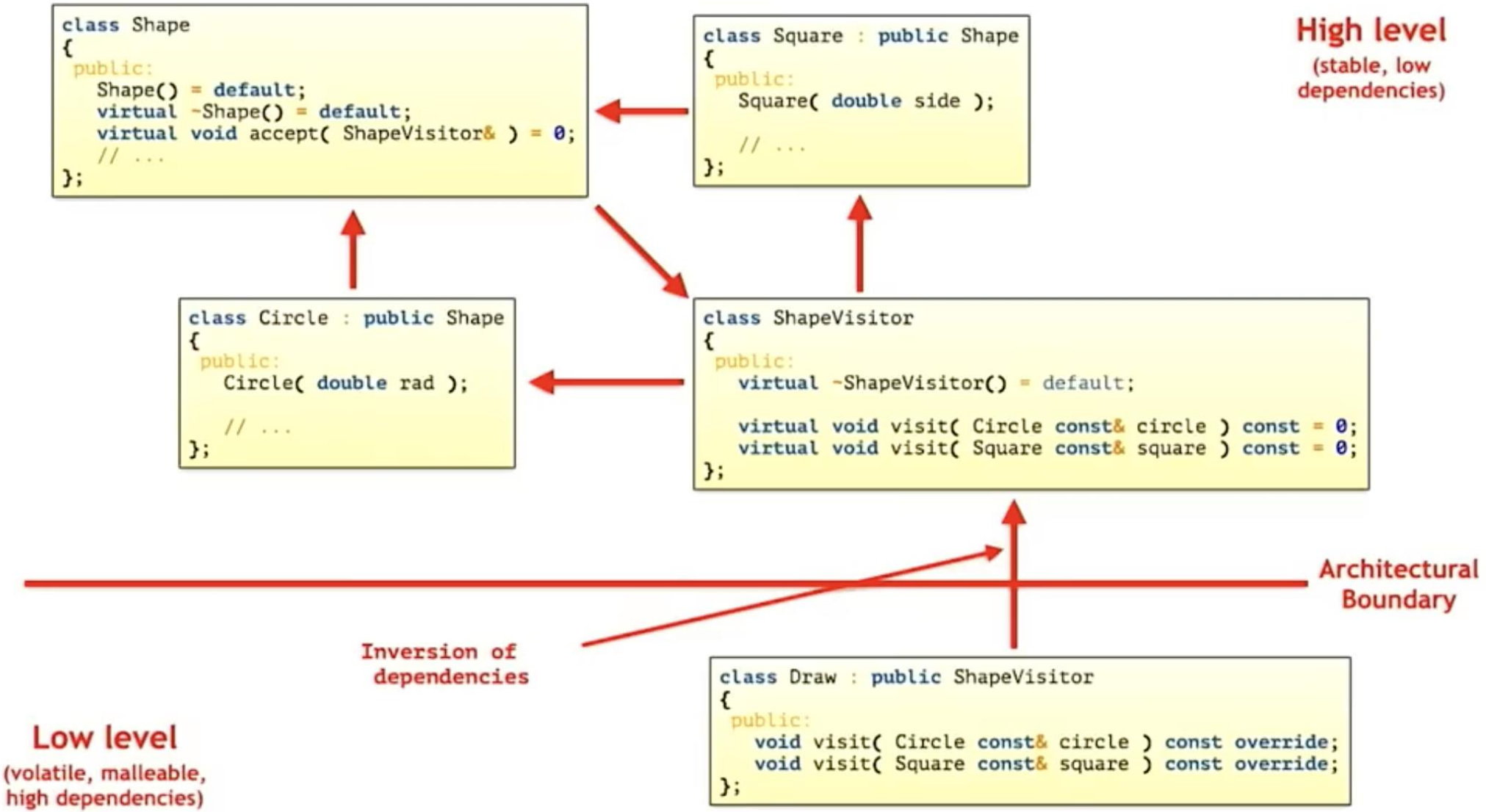


https://www.youtube.com/watch?v=PEcy1vYHb8A&t=662s

Visitor pros - cons

Pros	Cons
Operations can be added easily (OCP)	Makes it hard to add new types (shapes)
Hides implementation details of operations (SRP)	Operations are limited to public interface of types
	Base class required (God object)
	Heap allocation needed
	Memory management needed
	Requires virtual functions (extra indirection)
	Has a cyclic dependency !!

The classic Visitor pattern has cyclic dependency



How about a procedural solution

Code demo: https://godbolt.org/z/35Yjsh5hT

- Needs heap allocated objects
- Relies on switch statement
- Impossible to maintain

A modern solution: std::variant + std::visit

Code demo: https://godbolt.org/z/a4xGvchco

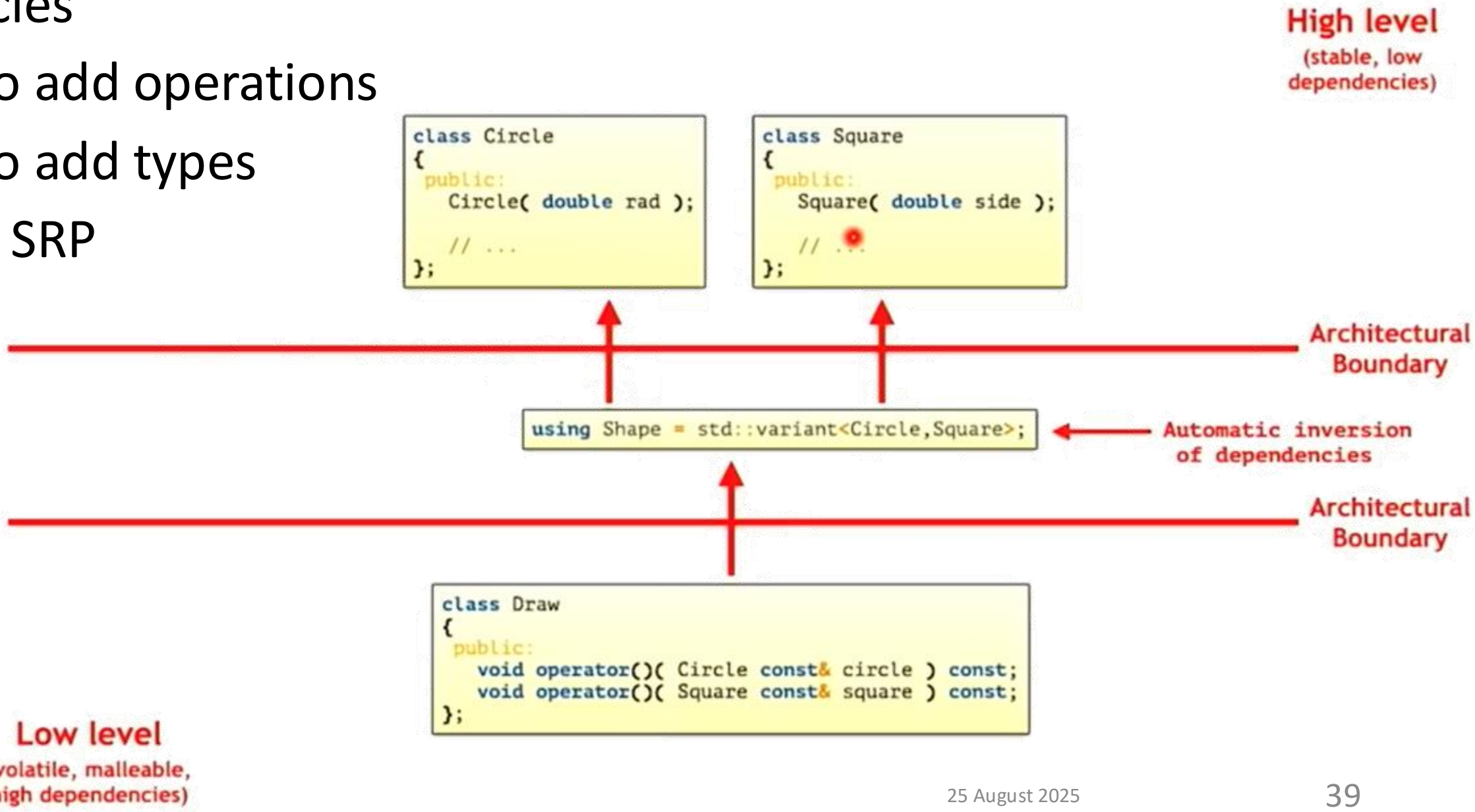
std::variant + std::visit

No cycles

Easy to add operations

Easy to add types

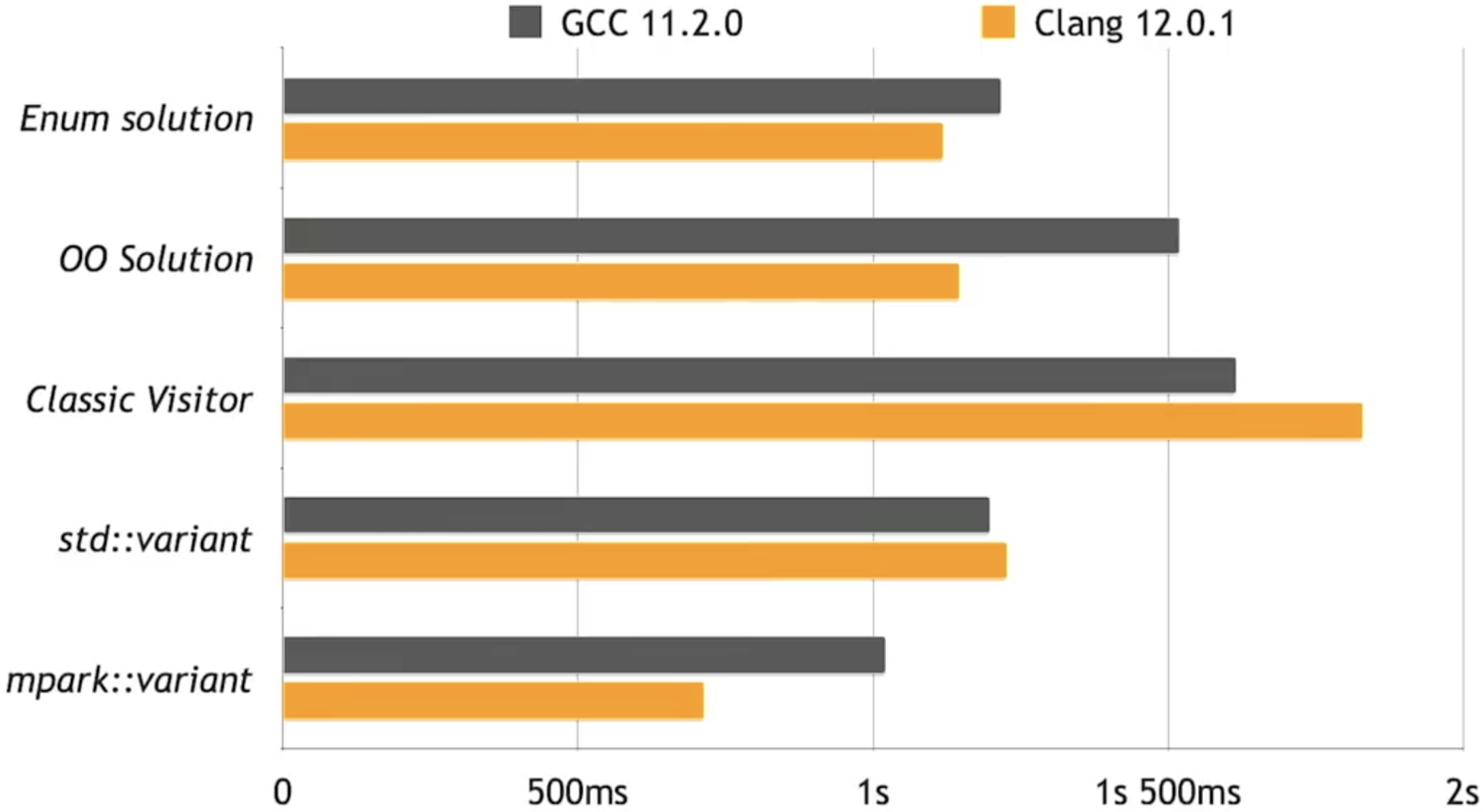
OCP + SRP



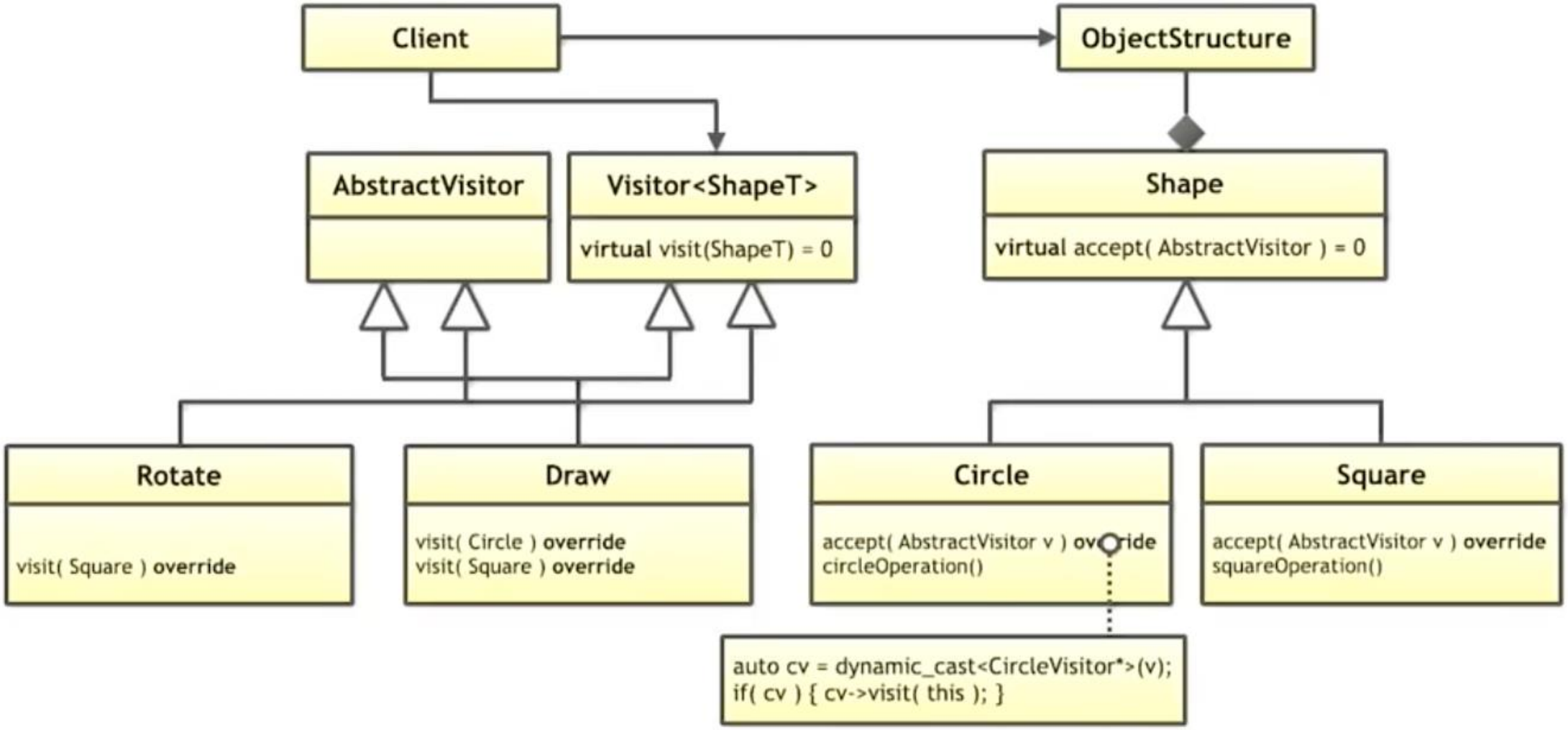
std::visit pros - cons

Pros	Cons
Operations can be added easily (OCP)	Mixing big + small types carries large overhead - Use proxy pattern to fix
Hides implementation details of operations (SRP)	std::variant is bad at hiding impl. details (info. hiding)
Easy to add types	
Operations can use all type details	
No manual heap alloc needed (auto destruction)	
No virtual functions	
No cyclic dependency	
Faster than OOP visitor	
Uses static polymorphism (faster, easier to optimize)	

Performance comparison

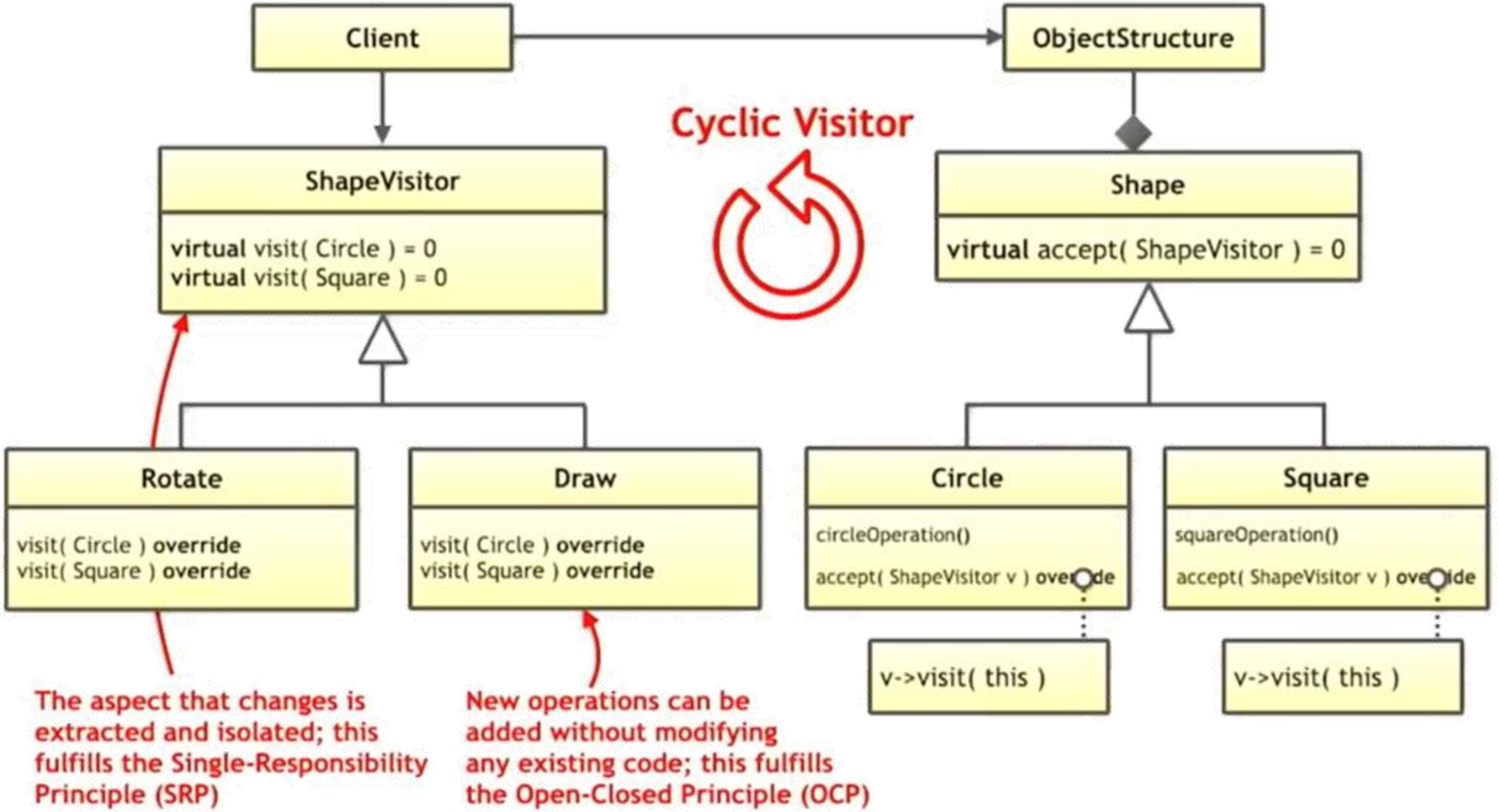


Acyclic OOP visitor pattern



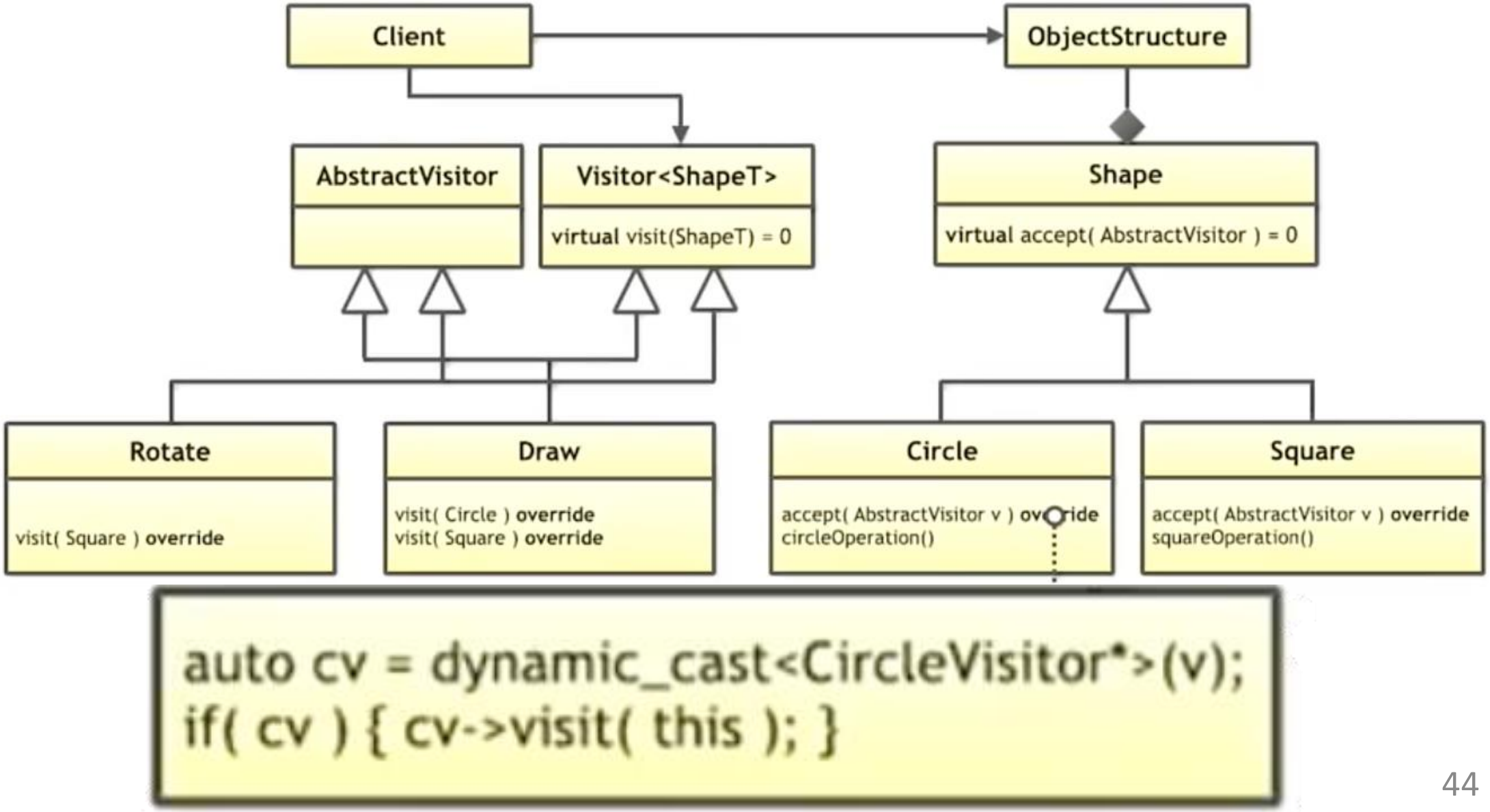
GoF visitor

Adds operations to a type hierarchy

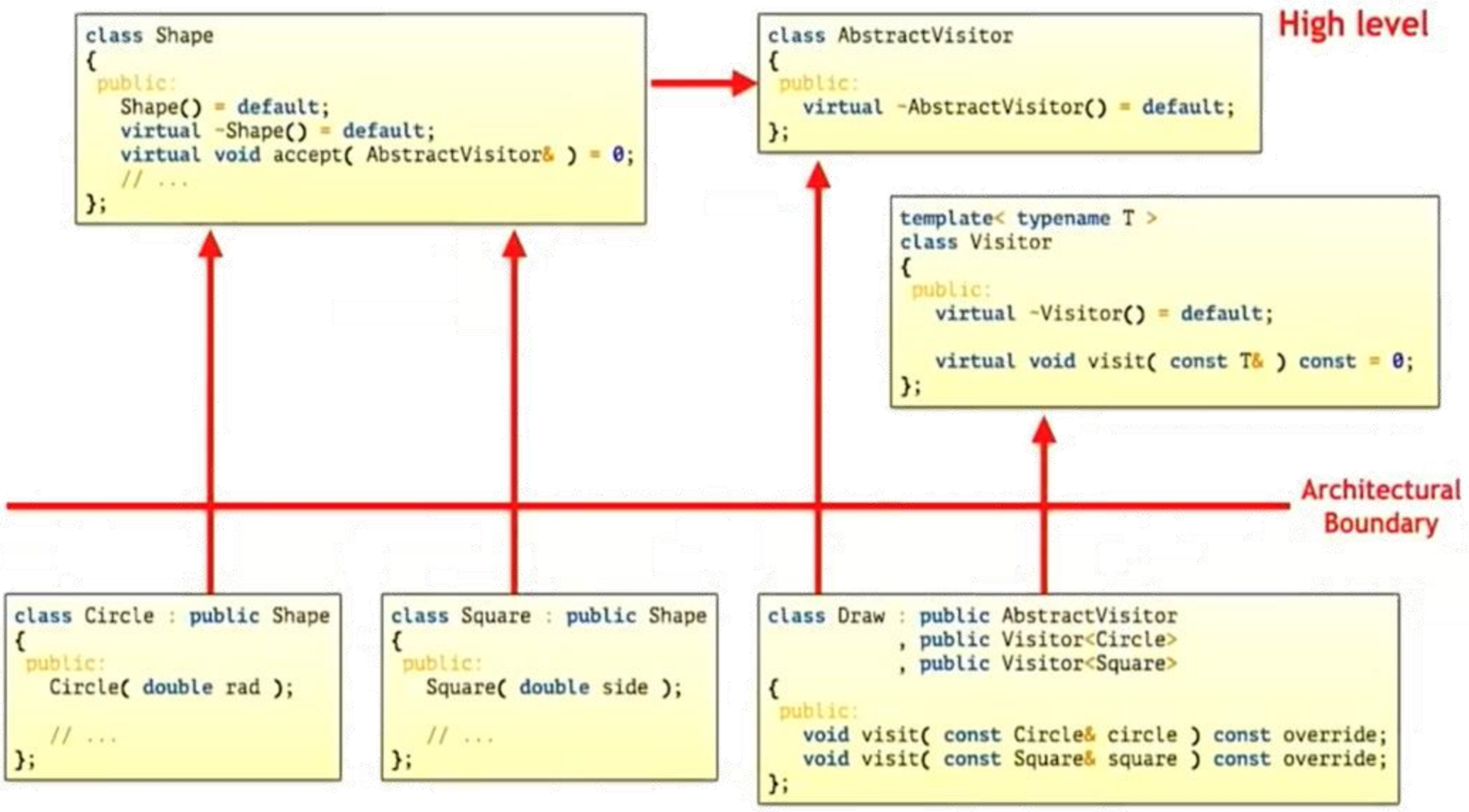


https://www.youtube.com/watch?v=PEcy1vYHb8A&t=662s

Acyclic OOP visitor pattern

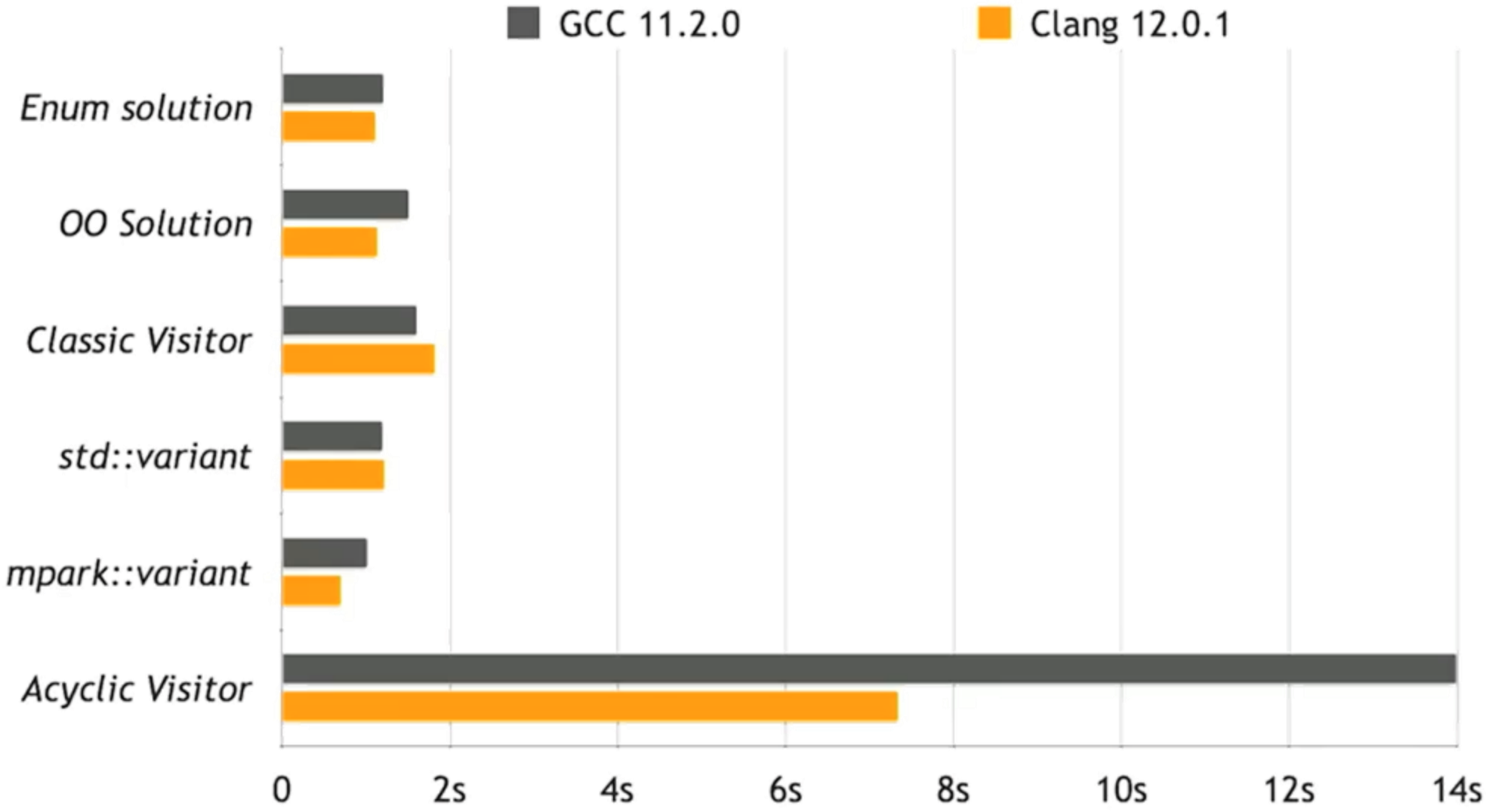


Acyclic visitor pattern



Acyclic OOP visitor

Drawback: slow, so very slow.



Recap

Visitor pattern

- Good if you want to add operations
- Bad if you want to add types

std::variant + std::visit

- Preferred in most scenarios

Acyclic visitor

- Also good for adding types
- Slow: use if overhead is not an issue
- Java: you have no choice